

11 Input/Output Overview

Most programs need to do either input (reading data) or output (writing data), or most frequently both, in order to do anything useful. The GNU C Library provides such a large selection of input and output functions that the hardest part is often deciding which function is most appropriate!

This chapter introduces concepts and terminology relating to input and output. Other chapters relating to the GNU I/O facilities are:

- Chapter 12 [Input/Output on Streams], page 273, which covers the high-level functions that operate on streams, including formatted input and output.
- Chapter 13 [Low-Level Input/Output], page 351, which covers the basic I/O and control functions on file descriptors.
- Chapter 14 [File System Interface], page 418, which covers functions for operating on directories and for manipulating file attributes such as access modes and ownership.
- Chapter 15 [Pipes and FIFOs], page 472, which includes information on the basic interprocess communication facilities.
- Chapter 16 [Sockets], page 477, which covers a more complicated interprocess communication facility with support for networking.
- Chapter 17 [Low-Level Terminal Interface], page 528, which covers functions for changing how input and output to terminals or other serial devices are processed.

11.1 Input/Output Concepts

Before you can read or write the contents of a file, you must establish a connection or communications channel to the file. This process is called *opening* the file. You can open a file for reading, writing, or both.

The connection to an open file is represented either as a stream or as a file descriptor. You pass this as an argument to the functions that do the actual read or write operations, to tell them which file to operate on. Certain functions expect streams, and others are designed to operate on file descriptors.

When you have finished reading from or writing to the file, you can terminate the connection by *closing* the file. Once you have closed a stream or file descriptor, you cannot do any more input or output operations on it.

11.1.1 Streams and File Descriptors

When you want to do input or output to a file, you have a choice of two basic mechanisms for representing the connection between your program and the file: file descriptors and streams. File descriptors are represented as objects of type `int`, while streams are represented as `FILE *` objects.

File descriptors provide a primitive, low-level interface to input and output operations. Both file descriptors and streams can represent a connection to a device (such as a terminal), or a pipe or socket for communicating with another process, as well as a normal file. But, if you want to do control operations that are specific to a particular kind of device, you must use a file descriptor; there are no facilities to use streams in this way. You must also

use file descriptors if your program needs to do input or output in special modes, such as nonblocking (or polled) input (see Section 13.15 [File Status Flags], page 402).

Streams provide a higher-level interface, layered on top of the primitive file descriptor facilities. The stream interface treats all kinds of files pretty much alike—the sole exception being the three styles of buffering that you can choose (see Section 12.20 [Stream Buffering], page 337).

The main advantage of using the stream interface is that the set of functions for performing actual input and output operations (as opposed to control operations) on streams is much richer and more powerful than the corresponding facilities for file descriptors. The file descriptor interface provides only simple functions for transferring blocks of characters, but the stream interface also provides powerful formatted input and output functions (`printf` and `scanf`) as well as functions for character- and line-oriented input and output.

Since streams are implemented in terms of file descriptors, you can extract the file descriptor from a stream and perform low-level operations directly on the file descriptor. You can also initially open a connection as a file descriptor and then make a stream associated with that file descriptor.

In general, you should stick with using streams rather than file descriptors, unless there is some specific operation you want to do that can only be done on a file descriptor. If you are a beginning programmer and aren't sure what functions to use, we suggest that you concentrate on the formatted input functions (see Section 12.14 [Formatted Input], page 319) and formatted output functions (see Section 12.12 [Formatted Output], page 295).

If you are concerned about portability of your programs to systems other than GNU, you should also be aware that file descriptors are not as portable as streams. You can expect any system running ISO C to support streams, but non-GNU systems may not support file descriptors at all, or may only implement a subset of the GNU functions that operate on file descriptors. Most of the file descriptor functions in the GNU C Library are included in the POSIX.1 standard, however.

11.1.2 File Position

One of the attributes of an open file is its *file position* that keeps track of where in the file the next character is to be read or written. On GNU systems, and all POSIX.1 systems, the file position is simply an integer representing the number of bytes from the beginning of the file.

The file position is normally set to the beginning of the file when it is opened, and each time a character is read or written, the file position is incremented. In other words, access to the file is normally *sequential*.

Ordinary files permit read or write operations at any position within the file. Some other kinds of files may also permit this. Files which do permit this are sometimes referred to as *random-access* files. You can change the file position using the `fseek` function on a stream (see Section 12.18 [File Positioning], page 333) or the `lseek` function on a file descriptor (see Section 13.2 [Input and Output Primitives], page 356). If you try to change the file position on a file that doesn't support random access, you get the `ESPIPE` error.

Streams and descriptors that are opened for *append access* are treated specially for output: output to such files is *always* appended sequentially to the *end* of the file, regardless

of the file position. However, the file position is still used to control where in the file reading is done.

If you think about it, you'll realize that several programs can read a given file at the same time. In order for each program to be able to read the file at its own pace, each program must have its own file pointer, which is not affected by anything the other programs do.

In fact, each opening of a file creates a separate file position. Thus, if you open a file twice even in the same program, you get two streams or descriptors with independent file positions.

By contrast, if you open a descriptor and then duplicate it to get another descriptor, these two descriptors share the same file position: changing the file position of one descriptor will affect the other.

11.2 File Names

In order to open a connection to a file, or to perform other operations such as deleting a file, you need some way to refer to the file. Nearly all files have names that are strings—even files which are actually devices such as tape drives or terminals. These strings are called *file names*. You specify the file name to say which file you want to open or operate on.

This section describes the conventions for file names and how the operating system works with them.

11.2.1 Directories

In order to understand the syntax of file names, you need to understand how the file system is organized into a hierarchy of directories.

A *directory* is a file that contains information to associate other files with names; these associations are called *links* or *directory entries*. Sometimes, people speak of “files in a directory”, but in reality, a directory only contains pointers to files, not the files themselves.

The name of a file contained in a directory entry is called a *file name component*. In general, a file name consists of a sequence of one or more such components, separated by the slash character (`/`). A file name which is just one component names a file with respect to its directory. A file name with multiple components names a directory, and then a file in that directory, and so on.

Some other documents, such as the POSIX standard, use the term *pathname* for what we call a file name, and either *filename* or *pathname component* for what this manual calls a file name component. We don't use this terminology because a “path” is something completely different (a list of directories to search), and we think that “pathname” used for something else will confuse users. We always use “file name” and “file name component” (or sometimes just “component”, where the context is obvious) in GNU documentation. Some macros use the POSIX terminology in their names, such as `PATH_MAX`. These macros are defined by the POSIX standard, so we cannot change their names.

You can find more detailed information about operations on directories in Chapter 14 [File System Interface], page 418.

11.2.2 File Name Resolution

A file name consists of file name components separated by slash (`'/'`) characters. On the systems that the GNU C Library supports, multiple successive `'/'` characters are equivalent to a single `'/'` character.

The process of determining what file a file name refers to is called *file name resolution*. This is performed by examining the components that make up a file name in left-to-right order, and locating each successive component in the directory named by the previous component. Of course, each of the files that are referenced as directories must actually exist, be directories instead of regular files, and have the appropriate permissions to be accessible by the process; otherwise the file name resolution fails.

If a file name begins with a `'/'`, the first component in the file name is located in the *root directory* of the process (usually all processes on the system have the same root directory). Such a file name is called an *absolute file name*.

Otherwise, the first component in the file name is located in the current working directory (see Section 14.1 [Working Directory], page 418). This kind of file name is called a *relative file name*.

The file name components `.` (“dot”) and `..` (“dot-dot”) have special meanings. Every directory has entries for these file name components. The file name component `.` refers to the directory itself, while the file name component `..` refers to its *parent directory* (the directory that contains the link for the directory in question). As a special case, `..` in the root directory refers to the root directory itself, since it has no parent; thus `/..` is the same as `/`.

Here are some examples of file names:

- `/a` The file named `a`, in the root directory.
- `/a/b` The file named `b`, in the directory named `a` in the root directory.
- `a` The file named `a`, in the current working directory.
- `/a/./b` This is the same as `/a/b`.
- `./a` The file named `a`, in the current working directory.
- `../a` The file named `a`, in the parent directory of the current working directory.

A file name that names a directory may optionally end in a `'/'`. You can specify a file name of `/` to refer to the root directory, but the empty string is not a meaningful file name. If you want to refer to the current working directory, use a file name of `.` or `./`.

Unlike some other operating systems, GNU systems don’t have any built-in support for file types (or extensions) or file versions as part of its file name syntax. Many programs and utilities use conventions for file names—for example, files containing C source code usually have names suffixed with `‘.c’`—but there is nothing in the file system itself that enforces this kind of convention.

11.2.3 File Name Errors

Functions that accept file name arguments usually detect these `errno` error conditions relating to the file name syntax or trouble finding the named file. These errors are referred to throughout this manual as the *usual file name errors*.

EACCES	The process does not have search permission for a directory component of the file name.
ENAMETOOLONG	This error is used when either the total length of a file name is greater than <code>PATH_MAX</code>, or when an individual file name component has a length greater than <code>NAME_MAX</code>. See Section 33.6 [Limits on File System Capacity], page 921. On GNU/Hurd systems, there is no imposed limit on overall file name length, but some file systems may place limits on the length of a component.
ENOENT	This error is reported when a file referenced as a directory component in the file name doesn't exist, or when a component is a symbolic link whose target file does not exist. See Section 14.6 [Symbolic Links], page 439.
ENOTDIR	A file that is referenced as a directory component in the file name exists, but it isn't a directory.
ELOOP	Too many symbolic links were resolved while trying to look up the file name. The system has an arbitrary limit on the number of symbolic links that may be resolved in looking up a single file name, as a primitive way to detect loops. See Section 14.6 [Symbolic Links], page 439.

11.2.4 Portability of File Names

The rules for the syntax of file names discussed in Section 11.2 [File Names], page 270, are the rules normally used by GNU systems and by other POSIX systems. However, other operating systems may use other conventions.

There are two reasons why it can be important for you to be aware of file name portability issues:

- If your program makes assumptions about file name syntax, or contains embedded literal file name strings, it is more difficult to get it to run under other operating systems that use different syntax conventions.
- Even if you are not concerned about running your program on machines that run other operating systems, it may still be possible to access files that use different naming conventions. For example, you may be able to access file systems on another computer running a different operating system over a network, or read and write disks in formats used by other operating systems.

The ISO C standard says very little about file name syntax, only that file names are strings. In addition to varying restrictions on the length of file names and what characters can validly appear in a file name, different operating systems use different conventions and syntax for concepts such as structured directories and file types or extensions. Some concepts such as file versions might be supported in some operating systems and not by others.

The POSIX.1 standard allows implementations to put additional restrictions on file name syntax, concerning what characters are permitted in file names and on the length of file name and file name component strings. However, on GNU systems, any character except the null character is permitted in a file name string, and on GNU/Hurd systems there are no limits on the length of file name strings.

12 Input/Output on Streams

This chapter describes the functions for creating streams and performing input and output operations on them. As discussed in Chapter 11 [Input/Output Overview], page 268, a stream is a fairly abstract, high-level concept representing a communications channel to a file, device, or process.

12.1 Streams

For historical reasons, the type of the C data structure that represents a stream is called `FILE` rather than “stream”. Since most of the library functions deal with objects of type `FILE *`, sometimes the term *file pointer* is also used to mean “stream”. This leads to unfortunate confusion over terminology in many books on C. This manual, however, is careful to use the terms “file” and “stream” only in the technical sense.

The `FILE` type is declared in the header file `stdio.h`.

FILE [Data Type]

This is the data type used to represent stream objects. A `FILE` object holds all of the internal state information about the connection to the associated file, including such things as the file position indicator and buffering information. Each stream also has error and end-of-file status indicators that can be tested with the `ferror` and `feof` functions; see Section 12.15 [End-Of-File and Errors], page 330.

`FILE` objects are allocated and managed internally by the input/output library functions. Don't try to create your own objects of type `FILE`; let the library do it. Your programs should deal only with pointers to these objects (that is, `FILE *` values) rather than the objects themselves.

12.2 Standard Streams

When the `main` function of your program is invoked, it already has three predefined streams open and available for use. These represent the “standard” input and output channels that have been established for the process.

These streams are declared in the header file `stdio.h`.

FILE * stdin [Variable]

The *standard input* stream, which is the normal source of input for the program.

FILE * stdout [Variable]

The *standard output* stream, which is used for normal output from the program.

FILE * stderr [Variable]

The *standard error* stream, which is used for error messages and diagnostics issued by the program.

On GNU systems, you can specify what files or processes correspond to these streams using the pipe and redirection facilities provided by the shell. (The primitives shells use to implement these facilities are described in Chapter 14 [File System Interface], page 418.)

Most other operating systems provide similar mechanisms, but the details of how to use them can vary.

In the GNU C Library, `stdin`, `stdout`, and `stderr` are normal variables which you can set just like any others. For example, to redirect the standard output to a file, you could do:

```
fclose (stdout);
stdout = fopen ("standard-output-file", "w");
```

Note however, that in other systems `stdin`, `stdout`, and `stderr` are macros that you cannot assign to in the normal way. But you can use `freopen` to get the effect of closing one and reopening it. See Section 12.3 [Opening Streams], page 274.

The three streams `stdin`, `stdout`, and `stderr` are not unoriented at program start (see Section 12.6 [Streams in Internationalized Applications], page 282).

12.3 Opening Streams

Opening a file with the `fopen` function creates a new stream and establishes a connection between the stream and a file. This may involve creating a new file.

Everything described in this section is declared in the header file `stdio.h`.

FILE * fopen (const char *filename, const char *opentype) [Function]
 Preliminary: | MT-Safe | AS-Unsafe heap lock | AC-Unsafe mem fd lock | See
 Section 1.2.2.1 [POSIX Safety Concepts], page 2.

The `fopen` function opens a stream for I/O to the file *filename*, and returns a pointer to the stream.

The *opentype* argument is a string that controls how the file is opened and specifies attributes of the resulting stream. It must begin with one of the following sequences of characters:

- ‘r’ Open an existing file for reading only.
- ‘w’ Open the file for writing only. If the file already exists, it is truncated to zero length. Otherwise a new file is created.
- ‘a’ Open a file for append access; that is, writing at the end of file only. If the file already exists, its initial contents are unchanged and output to the stream is appended to the end of the file. Otherwise, a new, empty file is created.
- ‘r+’ Open an existing file for both reading and writing. The initial contents of the file are unchanged and the initial file position is at the beginning of the file.
- ‘w+’ Open a file for both reading and writing. If the file already exists, it is truncated to zero length. Otherwise, a new file is created.
- ‘a+’ Open or create file for both reading and appending. If the file exists, its initial contents are unchanged. Otherwise, a new file is created. The initial file position for reading is at the beginning of the file, but output is always appended to the end of the file.